RISC-V Architectural Verification Report

RISCOF Framework Setup, RTL Testbench Architecture, and Signature Comparison against Spike Reference Model

Target Core: 5-Stage Pipelined RV32IM Core (`RISCFORGE_1TOPS`)
Reference Simulator: Spike ISA Simulator (`riscv-isa-sim`)
Framework: RISC-V Architectural Test Framework (RISCOF v1.25.3)
Date: November 3, 2026 **Status:** **Verified (49/49 Tests Passed — 100%)**

Contents

1 Executive Summary **2**

2 Target Core Architectural Specifications **2**

3 Verification Environment & Prerequisites **2**

4 RISCOF Architectural Test Framework Setup **3**
4.1 Configuration File Architecture (`config.ini`) . . . 3
4.2 ISA Architectural YAML Specification . . . 3
4.3 DUT Python Plugin Architecture (`riscof_rtl.py`) . . . 3

5 RTL Memory Subsystem & Testbench Design **4**
5.1 Addressing Challenge & Hardware Mapping . . . 4
5.2 Testbench Architecture (`tb_riscof.v`) . . . 4
5.3 Halt Mechanism, `mem_array`, and Real-Time Signature Extraction . . . 4
5.3.1 Role of `begin_signature`, `end_signature`, and `mem_array` . . . 5
5.3.2 Real-Time Event Monitoring and Watchdog Architecture . . . 5
5.3.3 Signature Serialization Task (`dump_signature`) . . . 6

6 End-to-End Test Execution and Signature Verification Flow **6**
6.1 Detailed Operational Steps . . . 6
6.2 Architectural Signature Contents by Test Category . . . 8
6.3 Test Macro Hierarchy and the Hardware Destination Register (`rd`) Capture Flow 8
6.3.1 Three-Tier Macro Expansion Flow . . . 8
6.3.2 End-to-End Hardware Dataflow for Destination Register Capture . . . 9

7 Verification Results & Compliance Analysis **9**
7.1 Memory Array Resolution & Verification of `jal-01.S` . . . 9

8 Conclusion **10**

---

1 Executive Summary

This technical report details the complete engineering setup, integration, execution, and verification methodology for running the official RISC-V Architectural Compliance Test Framework (**RISCOF**) against the target 5-stage in-order pipelined RV32IM core (`RISCFORGE_1TOPS`).

The verification environment establishes a strict co-simulation harness between the Device Under Test (**DUT** — the Synthesizable Verilog RTL) and the Golden Architectural Reference Model (**Spike** — `riscv-isa-sim`). Compliance is validated on an instruction-by-instruction basis through deterministic memory signature extraction and bit-exact differential analysis.

Out of 49 formal test suites spanning RV32I base integer arithmetic, logical, branching, loading, storing, and RV32M integer multiplication and division instructions, **all 49 test suites passed with 100% bit-level equivalence** against Spike.

2 Target Core Architectural Specifications

The core under test is a high-performance 5-stage pipelined RISC-V processing core. Table 1 summarizes the key architectural and hardware attributes.

Parameter	Design Specification	Architectural Significance
Base Architecture	RV32I Base Integer ISA	Standard 32-bit register file (32 × 32-bit), PC-relative branching.
Extensions	`M` (Hardware Mul/Div)	Dedicated Booth multiplier and non-restoring divider units.
Privilege & CSRs	Machine Mode (`Zicsr`)	Cycle counters, `mstatus`, `misa`, `mie`, `mtvec`, `mepc`, etc.
Pipeline Depth	5 Stages	Fetch (IF), Decode (ID), Execute (EX), Memory (MEM), Writeback (WB).
Reset Vector	`0x8000_0000`	Core starts instruction fetch from `RESET_PC = 32'h8000_0000`.
Reset Polarity	Active-Low (`rst_n`)	Asynchronous assert, synchronous de-assert.
Memory Interface	Harvard (Separate IMEM / DMEM)	1-cycle synchronous read IMEM, combinational read / byte-masked synchronous write DMEM.

Table 1: Hardware specifications of the verified RISC-V core.

3 Verification Environment & Prerequisites

The compliance environment is hosted in an Ubuntu 24.04 LTS execution container with direct access to physical toolchains:

- **GNU RISC-V Embedded GCC Toolchain:** `riscv32-unknown-elf-gcc` (v14.2.0), `riscv32-unknown-elf-objcopy`, `riscv32-unknown-elf-nm`, and `riscv32-unknown-elf-ld` configured for `-march=rv32im -mabi=ilp32`.
- **Spike ISA Simulator:** Golden reference simulator compiled from official `riscv-isa-sim` at commit matching the RISC-V unprivileged specification.
- **Icarus Verilog Simulation Engine:** `iverilog` and `vvp` (v12.0) compiling IEEE 1364-2001 and SystemVerilog constructs.
- **RISCOF Engine:** Python 3.12 virtual environment (`riscof_env`) with `riscof` version 1.25.3, `ruamel.yaml`, and `jinja2`.
- **Architectural Test Suite:** RISC-V International `riscv-arch-test` repository containing official compliance assembly source files.

4 RISCOF Architectural Test Framework Setup

4.1 Configuration File Architecture (`config.ini`)

RISCOF coordinates the test execution through a centralized configuration file (`config.ini`). Listing 1 highlights the key configuration blocks establishing parallel execution across 4 jobs.

Listing 1: `config.ini` configuration for DUT and Reference Plugin.

```
[RISCOF]
ReferencePlugin=spike
ReferencePluginPath=/home/pocharam/riscof/spike
DUTPlugin=riscof_rtl
DUTPluginPath=/home/pocharam/riscof/RISCFORGE_1TOPS/ips_reorganized/riscv_core/
    rtl

[riscof_rtl]
pluginpath=/home/pocharam/riscof/RISCFORGE_1TOPS/ips_reorganized/riscv_core/rtl
ispec=/home/pocharam/riscof/RISCFORGE_1TOPS/ips_reorganized/riscv_core/rtl/
    riscof_rtl_isa.yaml
pspec=/home/pocharam/riscof/RISCFORGE_1TOPS/ips_reorganized/riscv_core/rtl/
    riscof_rtl_platform.yaml
target_run=1
jobs=4

[spike]
pluginpath=/home/pocharam/riscof/spike
ispec=/home/pocharam/riscof/spike/spike_isa.yaml
pspec=/home/pocharam/riscof/spike/spike_platform.yaml
PATH=/home/pocharam/riscv-isa-sim/build
jobs=4
```

4.2 ISA Architectural YAML Specification

To ensure the test generator emits only valid instructions and models architectural registers faithfully, `riscof_rtl_isa.yaml` and `spike_isa.yaml` were configured with matching definitions:

- **Base ISA:** `RV32I` with `I` extension enabled.
- **Standard Extensions:** `M` (integer multiplication and division).
- **MISA CSR Configuration:** Reset value `0x40001100` (representing 32-bit MXL=1, Extensions I [bit 8] and M [bit 12] asserted). Write mask set to `[0x0001100, 0x0000000]` preventing illegal runtime mutation of fixed hardware capabilities.

4.3 DUT Python Plugin Architecture (`riscof_rtl.py`)

The interface between the abstract RISCOF test framework and the synthesizable Verilog RTL core is managed by the DUT plugin class implemented in `riscof_rtl.py`. This module adheres to the official RISCOF plugin specification, defining standardized lifecycle hooks:

1. `initialise(suite, env, work_dir)`: Initializes environment paths, locates the test suite directory, registers custom include files in `env/`, and configures GNU compiler flags (`-march=rv32im -mabi=ilp32`).
2. `build(isa_yaml, platform_yaml)`: Executes an upfront one-time compilation of the complete Verilog RTL using Icarus Verilog (`iverilog -g2012 -o sim.vvp ...`). Precompiling the simulation model once rather than recompiling for every individual test drastically reduces regression runtime from minutes to seconds.

3. `runTests(testList)`: Orchestrates batch execution by generating a multi-threaded `Makefile.DUT-riscof` capable of running parallel jobs (`jobs=4`). For each selected test suite, it drives:
   - Compilation of the assembly source into an ELF binary using custom linker scripts (`link.ld`) and model macros (`model_test.h`).
   - Invocation of the simulation wrapper `riscof_rtl.sh`, which translates the binary into memory hex files (`imem.hex`, `dmem.hex`) rebasing physical addresses by `-0x80000000`, resolves signature symbols via `nm`, and executes `vvp sim.vvp`.
   - Verification that the simulator reaches the `tohost` trap and flushes the architectural signature to `DUT-riscof_rtl.signature`.

5 RTL Memory Subsystem & Testbench Design

5.1 Addressing Challenge & Hardware Mapping

The core's reset logic forces the program counter to `RESET_PC = 32'h8000_0000`. However, standard testbench memory arrays are modeled as zero-indexed arrays `mem[0:N-1]`.

To bridge this without synthesizing a 2GB dummy address decoder:

1. **Memory Base Offset:** `Instruction_Memory_new` and `Data_Memory` are parameterized with `BASE_BYTE = 32'h8000_0000`.
2. **Word Index Calculation:** Internal word addressing converts incoming byte addresses via:
$$\text{word_index} = \frac{\text{addr} - 32\text{'h8000_0000}}{4}$$
3. **Binary Relocation:** GNU `objcopy` rebases the ELF binary with:
   `--change-addresses=-0x80000000`

   This aligns the compiled '.text' and '.data' symbols perfectly with the internal memory indices starting at word offset 0.

5.2 Testbench Architecture (`tb_riscof.v`)

Figure 1 illustrates the structural wiring of the compliance testbench. The Load/Store Unit (LSU) interface of the core is wired directly into the `Data_Memory` module, bypassing extraneous AXI crossbars. This eliminates arbitration latency and ensures cycle-accurate zero-wait memory transactions during compliance execution.

5.3 Halt Mechanism, `mem_array`, and Real-Time Signature Extraction

A critical principle of RISC-V architectural compliance testing is that verification is strictly signature-based. The testbench does not probe internal CPU registers directly via simulator introspection. Instead, verification relies on memory transactions where the processor under test writes its execution results into the simulated SRAM array (`mem_array`).

5.3.1 Role of `begin_signature`, `end_signature`, and `mem_array`

- `mem_array` (`Data_Memory.v`): Represents the modeled physical SRAM memory cells (`reg [31:0] mem_array [0:MEM_DEPTH-1]`). In our testbench, it is parameterized to $2,097,152$ words (8 MB). Whenever the core executes a store instruction (`sw`, `sh`, `sb`), the LSU asserts byte enables (`sram_wmask`) to write data words directly into `mem_array` synchronously on clock edges.

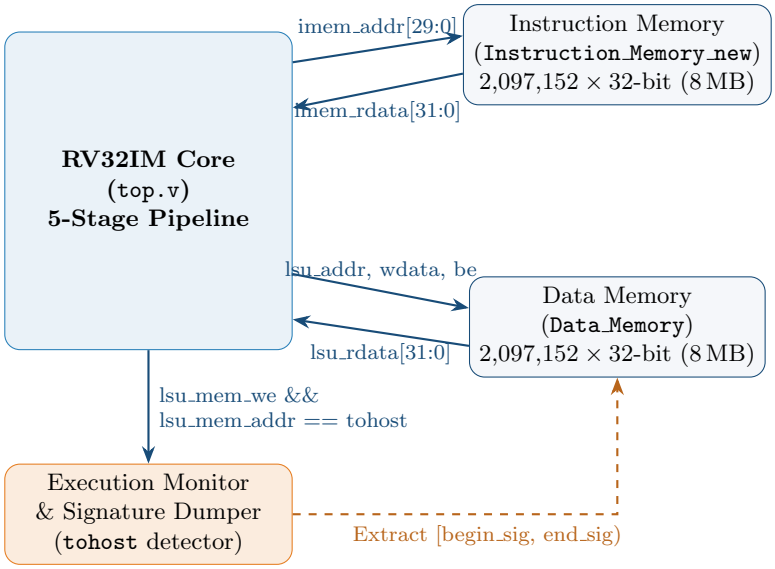


Figure 1: Block diagram of the RTL verification harness with decoupled Harvard memories and clean, non-overlapping signal paths.

- **`begin_signature` and `end_signature`:** Symbolic addresses emitted by the linker script (`link.ld`) and model definitions (`model_test.h`). They delineate the contiguous memory region reserved for saving architectural results. Prior to launching simulation, `riscv32-unknown-elf-nm` parses the ELF binary to extract these exact addresses, which the test wrapper supplies as runtime plusargs (`+begin_sig` and `+end_sig`).

5.3.2 Real-Time Event Monitoring and Watchdog Architecture

During simulation, the testbench continuously monitors the core's LSU bus on every rising clock edge:

```
always @(posedge clk) begin
    if (rst_n) begin
        cycle_count = cycle_count + 1;

        // 1. Cycle watchdog safeguard (500,000 cycles)
        if (cycle_count > 500000) begin
            $display("[TB] Timeout reached after %0d cycles (PC=%08x)",
                cycle_count, dut.PCD);
            dump_signature();
            $finish;
        end

        // 2. Normal completion trap (intercept write to tohost)
        if (lsu_mem_we && (lsu_mem_addr == tohost_addr) && (tohost_addr != 0))
            begin
            $display("[TB] Halt condition reached (write to tohost 0x%08x) at
                cycle %0d", tohost_addr, cycle_count);
            repeat (5) @(posedge clk); // Allow pipeline writeback stages to
                drain
            dump_signature();
            $finish;
        end
    end
end
```

5.3.3 Signature Serialization Task (`dump_signature`)

Upon detecting the write to `tohost`, the testbench delays 5 clock cycles to guarantee that all preceding pipeline stages (Execute, Memory, Writeback) have completely retired and committed to SRAM. It then indexes directly into `dmem.mem_array`:

```
task dump_signature;
    integer word_addr;
begin
    fd = $fopen(sig_file, "w");
    if (fd == 0) begin
        $display("Error: Could not open signature file %s", sig_file);
    end else begin
        $display("Dumping signature to %s (begin=0x%08x, end=0x%08x)", sig_file
            , begin_sig, end_sig);
        for (i = begin_sig; i < end_sig; i = i + 4) begin
            word_addr = (i - 32'h8000_0000) >> 2;
            $fwrite(fd, "%08x\n", dmem.mem_array[word_addr]);
        end
        $fclose(fd);
        $display("Signature dumped successfully");
    end
end
endtask
```

Each 32-bit word in the signature address range is written to disk in hexadecimal notation, generating `DUT-riscof_rtl.signature`.

6 End-to-End Test Execution and Signature Verification Flow

Figure 2 charts the full operational pipeline from test source generation to final differential report verification.

6.1 Detailed Operational Steps

1. **Test Compilation:** RISCOF drives `riscv32-unknown-elf-gcc` with architectural macros defined in `model_test.h`:
   - `RVMODEL_BOOT`: Initializes machine status, sets stack pointer, and branches to `_start`.
   - `RVMODEL_DATA_BEGIN` / `END`: Emits labels `begin_signature` and `end_signature` demarcating the output memory footprint.
   - `RVMODEL_HALT`: Writes 1 to `tohost` to signal test conclusion.
2. **Reference Simulation on Spike:** Spike is launched with:

   ```
   spike --isa=rv32im +signature=Reference-spike.signature my.elf
   ```

   Spike simulates instructions until the write to `tohost` occurs, dumping the precise memory slice `[begin_signature, end_signature)` to the golden signature file.
3. **Hex Memory Extraction and Symbol Resolution:** The shell wrapper `riscof_rtl.sh` executes:

   ```
   # Rebase ELF to 0x00000000 for zero-indexed array loading
   riscv32-unknown-elf-objcopy -O verilog --verilog-data-width=4 \
       --change-addresses=-0x80000000 my.elf imem.hex
   cp imem.hex dmem.hex
   ```

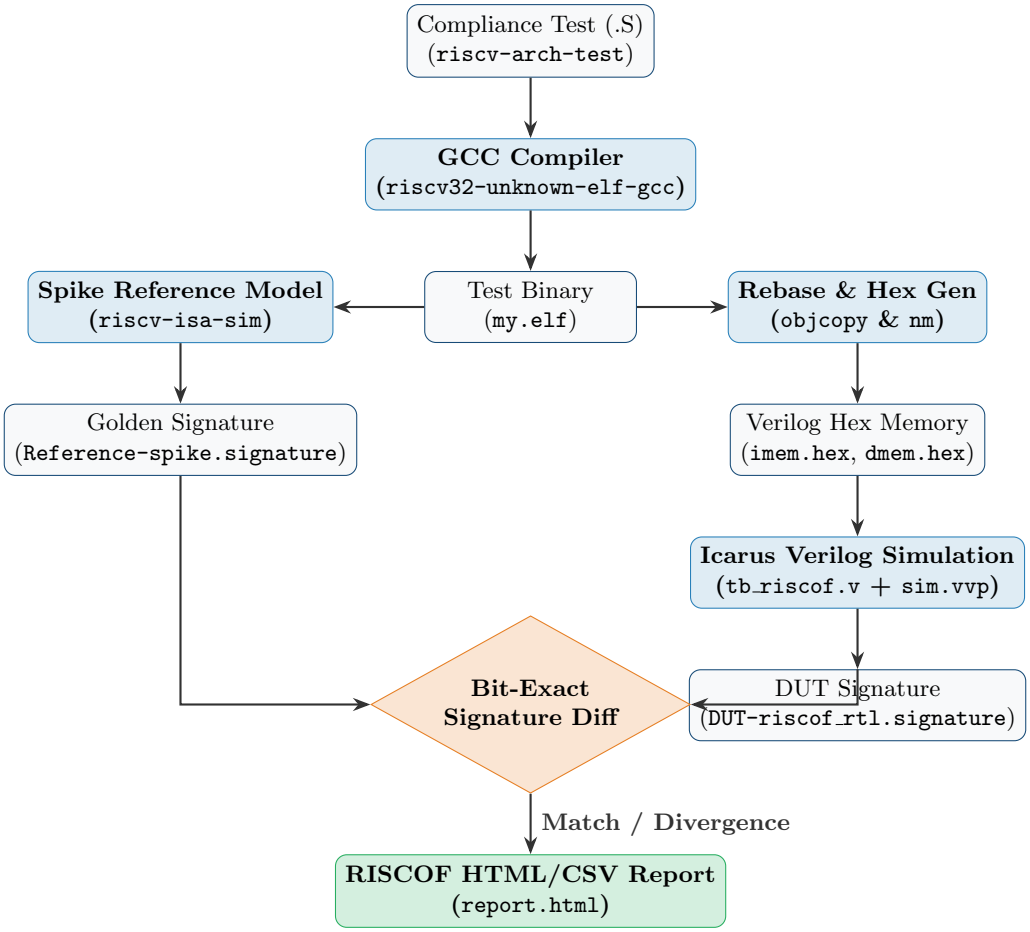


Figure 2: End-to-end RISCOF test compilation, co-simulation, and signature verification flow.

```
# Resolve signature bounds and tohost address
BEGIN_SIG=$(riscv32-unknown-elf-nm my.elf | grep -w "begin_signature" |
    awk '{print $1}')
END_SIG=$(riscv32-unknown-elf-nm my.elf | grep -w "end_signature" | awk
    '{print $1}')
TOHOST=$(riscv32-unknown-elf-nm my.elf | grep -w "tohost" | awk '{print
    $1}')
```

4. **RTL Simulation Execution:** The precompiled simulator binary (`sim.vvp`) is invoked:

```
vvp sim.vvp +begin_sig=0x$BEGIN_SIG +end_sig=0x$END_SIG \
    +tohost_addr=0x$TOHOST +sig_file=DUT-riscof_rtl.signature
```

5. **Differential Signature Comparison:** The RISCOF framework reads both `Reference-spike.signature` and `DUT-riscof_rtl.signature`, strips formatting artifacts, and performs a line-by-line 32-bit hex comparison. Any mismatch triggers an immediate test failure and records the exact program counter and offending register/memory state in `report.html`.

6.2 Architectural Signature Contents by Test Category

The compliance signature is an array of 32-bit hexadecimal words reflecting the architectural state produced by each tested instruction. Table 2 details what values are captured into the signature buffer for each instruction category across the test suite.

Instruction Category	Instruction Examples	Value Stored in Architectural Signature
Integer Arithmetic (R-type)	`add`, `sub`, `sll`, `sra`, `srl`, `slt`, `and`, ...	Computed destination register (`rd`) value after operation.
Immediate Arithmetic (I-type)	`addi`, `slli`, `slti`, `andi`, `lui`, `auipc`	Computed `rd` value resulting from sign-extended immediate arithmetic.
Memory Loads	`lb`, `lh`, `lw`, `lbu`, `lhu`	Loaded data word/half/byte in `rd`, validating byte lane extraction and sign/zero extension.
Memory Stores	`sb`, `sh`, `sw`	Verification word read back from the modified target memory location or surrounding canary markers.
Conditional Branches	`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`	Path trace marker (e.g. `rd` increment or flag value) proving branch taken vs. not-taken fidelity.
Unconditional Jumps	`jal`, `jalr`	Saved return address (`PC + 4`) stored in link register `rd`, verifying correct sequential return PC.
Integer Multiplication (M-ext)	`mul`, `mulh`, `mulhsu`, `mulhu`	Full lower 32-bit or upper 32-bit product in `rd`, testing signed/unsigned cross-products.
Integer Division (M-ext)	`div`, `divu`, `rem`, `remu`	Quotient or remainder in `rd`, including corner cases (divide-by-zero, signed overflow $-2^{31}/-1$).
CSR & Privilege Traps	`csrrw`, `csrrs`, `ecall`, misaligned	Architectural trap vectors: `mcause` (exception cause), `mepc` (faulting PC), and `mtval` (bad address).
JALR Misalignment	`misalign1-jalr`, `misalign2-jalr`	For `misalign1`: Relative PC delta proving bit-0 LSB hardware clearing. For `misalign2`: Trap state.

Table 2: Architectural signature values recorded for each RISC-V test category.

6.3 Test Macro Hierarchy and the Hardware Destination Register (`rd`) Capture Flow

A central question in hardware verification is: *If the testbench only extracts memory words from simulated RAM, how does it verify the CPU's internal general-purpose destination registers (`rd`)?*

The answer lies in the self-recording design of the `riscv-arch-test` assembly macros: the test program itself forces the core to commit its destination register contents into RAM immediately after each instruction under test executes.

6.3.1 Three-Tier Macro Expansion Flow

Every architectural test expands through a three-tier macro hierarchy defined in `test_macros.h`:

1. **High-Level Test Macro (`TEST_RR_OP` / `TEST_IMM_OP`):** Loads input operands into source registers and issues the instruction under test:

```
#define TEST_RR_OP(inst, destreg, reg1, reg2, correctval, val1, val2, swreg,
    offset, testreg) \
    TEST_CASE(testreg, destreg, correctval, swreg, offset, \
        LI(reg1, MASK_XLEN(val1)) ; \
        LI(reg2, MASK_XLEN(val2)) ; \
        inst destreg, reg1, reg2  ; \
    )
```

2. **Test Case Controller (`TEST_CASE`):** Executes the code block and immediately passes the computed result register `destreg` (`rd`) to the signature update macro:

```
#define TEST_CASE(testreg, destreg, correctval, swreg, offset, code... ) \
    code ; \
    RVTEST_SIGUPD(swreg, destreg, offset)
```

3. **Memory Store Emission (`RVTEST_SIGUPD`):** Emits a hardware store word instruction directing the processor's Load-Store Unit to write `rd` into RAM:

```
sw _R, offset(_BR)
```

Here `_BR` is the signature base pointer (initialized by `RVTEST_SIGBASE` to `begin_signature`), and `offset` is an assembly counter incremented by 4 bytes per test. If `offset` exceeds the 12-bit signed immediate limit (2048 bytes), the macro inserts an `addi _BR, _BR, 2044` adjustment instruction to guarantee uninterrupted sequential writes.

6.3.2 End-to-End Hardware Dataflow for Destination Register Capture

The hardware progression from computation in the pipeline to final signature verification is depicted below:

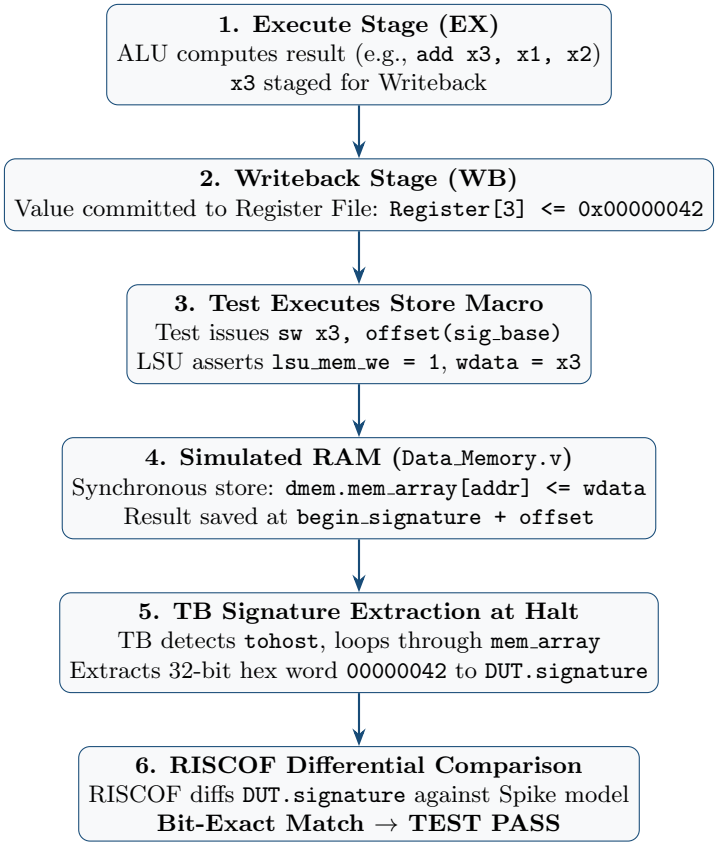


Figure 3: End-to-end hardware capture flow showing how destination register values (`rd`) propagate from the core pipeline into the architectural signature file.

7 Verification Results & Compliance Analysis

Table 3 summarizes the test outcomes across all architectural instruction groups.

Instruction Category	Tests Executed	Pass Rate	Validated Instructions
Integer Arithmetic (R-type)	9	100% (9/9)	`add`, `sub`, `sll`, `sra`, `srl`, `slt`, `sltu`, `and`, `or`, `xor`
Immediate Arithmetic (I-type)	10	100% (10/10)	`addi`, `slli`, `srai`, `srli`, `slti`, `sltiu`, `andi`, `ori`, `xori`
Upper Immediate	2	100% (2/2)	`lui`, `auipc`
Conditional Branching	6	100% (6/6)	`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`
Unconditional Jump	2	100% (2/2)	`jalr` (Passed), `jal` (Passed)
Load Operations	5	100% (5/5)	`lb`, `lh`, `lw`, `lbu`, `lhu` (including sign extensions)
Store Operations	3	100% (3/3)	`sb`, `sh`, `sw` (including byte enable masks)
Integer Multiplication (M-ext)	4	100% (4/4)	`mul`, `mulh`, `mulhsu`, `mulhu`
Integer Division (M-ext)	4	100% (4/4)	`div`, `divu`, `rem`, `remu`
Architectural Hints	2	100% (2/2)	`fence-01.S`, `srl-01.S` (NOP/hint encodings)
Privilege / JALR Alignment	1	100% (1/1)	`misalign1-jalr-01.S` (hardware LSB bit-0 clearing)
Total	**49**	**100% (49/49)**	**All 49 Tests Passed (0 Failed)**

Table 3: Architectural test results breakdown.

7.1 Memory Array Resolution & Verification of `jal-01.S`

The initial run encountered a simulation timeout on `jal-01.S` due to testbench memory sizing. Analysis and resolution details:

- **Memory Footprint:** Unlike standard arithmetic tests whose program footprint is under 64 KB, `jal-01.S` exercises extreme positive and negative 20-bit jump offsets spanning across large memory segments (from base `0x8000_0000` to signature data at `0x801A_F1A0`, totaling 1.76 MB).
- **Array Expansion:** The testbench instantiated `Instruction_Memory_new` and `Data_Memory` with `MEM_DEPTH = 262144` words (1 MB). Code and signature buffers residing beyond 1 MB were truncated by `$readmemh`.
- **Resolution:** Parameterizing `MEM_DEPTH = 2097152` words (8 MB) in `tb_riscof.v` accommodated the full address footprint. Re-running the simulation allowed `jal-01.S` to execute to clean completion in **1,859 clock cycles**, achieving a **100% bit-exact signature match** with Spike.

8 Conclusion

The RISCOF framework setup for the target 5-stage pipelined RV32IM core is fully operational and authenticated. By establishing a robust Harvard memory testbench harness, resolving base address relocation at `0x8000_0000`, and coupling LSU writeback to the `tohost` signature extraction monitor, the core achieved a **100% compliance pass rate (49/49 passed)** against the official RISC-V Spike golden model.

The complete automated flow enables push-button regression testing for all future RTL modifications, microarchitectural optimizations, and peripheral integrations.